

CO2 AT-1

1) Linked list operations:-

1. Problem Understanding and datastructure Selection :-

- Data Structure:- List ADT implemented as a singly linked list with a pointer
- Key operations:-
 - append(val) : Appends an element at the end of the list without moving.
 - move-to-start() : Moves curr cursor to the beginning of the list
 - next() : Moves curr cursor to the next element
 - insert(val) : Inserts an element at the current cursor position

2. Solution and Design:-

Operations on List L2.

1. L2.append(10) → List : [10]
2. L2.append(20) → list : [10, 20]
3. L2.append(15) → list : [10, 20, 15]
4. L2.move to start() → curr points to 10
5. L2.insert(39) → Inserts [39, 10, 20, 15]
6. L2.next() → Moves curr to 10
7. L2.insert(12) → Inserts 12 at current position:
 - Final Configuration (L2):
Head → 39 → 12 (curr) → 10 → 20 → 15 → NULL

3. Analytical Summary:-

- Complexity: Append takes $O(1)$ time if a tail pointer is maintained ($O(N)$ otherwise). Insert, move to start, and next operations execute in $O(1)$ time

2) Binary Search Algorithm analysis:-

1. Problem Understanding:

- Data Structure: Sorted array of size $N=10$.
- Array Indexing (0 to 9):

Index [0:66, 1:111, 2:212, 3:323, 4:432, 5:543, 6:654, 7:765, 8:876, 9:987]

- Formula $mid = \left\lfloor \frac{low + high}{2} \right\rfloor$

2. Execution trace for Cases:-

Case 1: Key Element = 11

- Iteration 1:

- $Low = 0, high = 9 \rightarrow mid = \left\lfloor \frac{0+9}{2} \right\rfloor = 4$

- Array [4] = 432. Since $11 < 432$,
update $high = mid - 1 = 3$.

- Iteration 2:
- Low = 0, high = 3 $\Rightarrow$ mid = $\left\lfloor \frac{0+3}{2} \right\rfloor = 1$
- Array[1] = 111. Match found!
- Result: Found at index 1 (Position 2).

Case 2: Key element = 888

- Iteration 1:
- Low = 0, high = 9 $\Rightarrow$ mid = 4 $\Rightarrow$ n
- Array[4] = 432 Since 888 > 432, update
low = mid + 1 = 5

• Iteration 2:

- Low = 5, high = 9 $\Rightarrow$ mid = $\left\lfloor \frac{5+9}{2} \right\rfloor = 7$
Since 888 > 649, update
low = mid + 1 = 8

• Iteration 3:

- Low = 8, high = 9 $\Rightarrow$ mid = $\left\lfloor \frac{8+9}{2} \right\rfloor = 8$
Since 888 > 777, update
low = mid + 1 = 9

• Iteration 4:

- Low = 9, high = 9 $\Rightarrow$ mid = 9 $\Rightarrow$ Array[9]
Match found!

- Result: Found at index 9

Case 3: key element = 999

- Iteration 1:

Low = 0, high = 9, mid = 4 $\Rightarrow$ Array [4] = 432

999 > 432 $\Rightarrow$ low = 5.

- Iteration 2:

Low = 5, high = 9, mid = 7 $\Rightarrow$ Array [7] = 649

999 > 649 $\Rightarrow$ low = 8

- Iteration 3:

low = 8, high = 9, mid = 8 $\Rightarrow$ Array [8] = 777

999 > 777 $\Rightarrow$ low = 9

- Iteration 4:

low = 9, high = 9, mid = 9 $\Rightarrow$ Array [9] = 888

999 > 888 $\Rightarrow$ low = 10

Termination: low (10) > high (9) $\Rightarrow$

Space exhausted

- Result: Element not found

3. Complexity Analysis:-

- Time Complexity: $O(\log_2 N)$ average & worst case,
 $O(1)$ best case

- Space Complexity: $O(1)$ auxiliary space for iterative search

LIFO Data Structure :-

1) Problem Understanding:-

- Data Structure: Array based stack (Size = 5, Index: 0 to 4)
- Pointer: top (Initialized to -1 for an empty stack)

2) Step-by-step operation trace:-

Push 45	[45, -, -, -, -]	0	45 top value
Push 22	[45, 22, -, -, -]	1	22 top value
Push 11	[45, 22, 11, -, -]	2	11 top value
Push 77	[45, 22, 11, 77, -]	3	77 top value
Removes 77	[45, 22, 11, , -]	2	77 top value
Push 44	[45, 22, 11, 44, -]	3	44 top value
Push 33	[45, 22, 11, 44, 33]	4	33 top value
Removes 33	[45, 22, 11, 44,]	3	44 top value
Removes 44	[45, 22, 11, ,]	2	11 top value
Removes 11	[45, 22, , ,]	1	22 top value

Analysis:-

- ⇒ Push inserts an element at the top of the Stack
- ⇒ Pop removes the most recently inserted element
- ⇒ The stack follows the LIFO
- ⇒ Push and pop operations required $O(1)$ time complexity

4) Linear Infix Expressions:-

$$(a+b) + c * (d+e) + f * (g+h)$$

Expression procedu	Prefix	Pattern
a	a	a
b	b	b
(a+b)	+ab	ab+
d	d	d
e	e	e
(d+e)	+de	de+
c * (d+e)	c +de	cde+
(a+b) + c * (d+e)	+ +ab c +de	ab + cde + +
f	f	f
(a+b) + c * (d+e) + f	+ +ab c +de f	ab + cde + + f
g	g	g
h	h	h
(g+h)	+gh	gh+
(a+b) + c * (d+e) + f * (g+h)	+ + +ab c +de f +gh	ab + cde + + f + gh +

Analysis:-

→ Prefix rotation doesn't require parameter

→ Prefix rotation also eliminate parenthesis

Result:-

The given Infix expression was successfully converted into:

Pre fix : * + + + ab + c + d e + f g h

Post fix : ab + cde + f + g h + *

Problem Understanding:-

The problem compare a simple Queue (FIFO) with a priority Queue in a ticketing system. The aim is - to analyze how giving priority to VIP customers affect service order, fairness, waiting time and computational.

Data Structure Selection:-

Data Structure:- Queue

- Simple Queue (FIFO) :- Customers are served in the order they arrive.
- Priority Queue :- Customers with higher priority (VIP) are served before normal customers.

Justification:-

- Queue is suitable for ticket booking system.
- Priority Queue efficiently handle urgent are VIP customer.

Algorithm Design:-

Simple Queue (FIFO):

1. Customer enters the queue.
2. Customer is added at the rear.
3. Customer at the front is served first.
4. Repeat until the queue becomes empty.

Priority Queue :-

1. Customer enters the queue.
2. Check customer priority (VIP & Normal)
3. Insert according to priority.
4. Serve the highest-priority customer first.
5. Repeat until the queue becomes empty.

Advantages:-

- VIP or emergency customers receive faster service.
- Reduce waiting time for high-priority customers.
- Suitable for real-time application.

Result:-

- A Simple Queue provides fair service by following the FIFO principle, making it suitable for normal ticketing system.
- A priority queue improves service for VIP customers by serving them 1st, reducing their repeating time.
- Therefore the choice depends on whether fairness or priority-based service.